第4回：断面配置・押し出し・メッシュ生成のC#実装例

詳細解説 と 宿題の問題・解答 の固定例A〜Dを、7題材・153件の実行時チェックで確かめる.NET 10コンソール教材。

断面の点を配置し、実際に8頂点と12三角形のインデックスを生成する。`double` のタプルで成分式を追い、配置・内積・外積・行列変換を `System.Numerics` の `float` とも照合する。外部NuGet、CADライセンス、GUI、他セッションのプロジェクトへの依存はない。入力は固定値で、引数・標準入力・ファイル出力は使わない。

必要環境

- **.NET 10 SDK**。対象は `net10.0`。ランタイムだけではビルドできない。
- Visual Studioでは **Visual Studio 2026（18.0以上、Windows）**、「.NETデスクトップ開発」ワークロード、使用するVisual Studioに対応する.NET 10 SDKを使う。公式の導入・対応表 を参照。
- CLIでは.NET 10 SDKを導入したWindows・Linux・macOSのコンソールから実行できる。

Visual Studioで開いて実行する

1. 「プロジェクトやソリューションを開く」で ModelingExamples.sln を開く。従来形式の `.sln` と `.csproj` が同じディレクトリにある。
2. プロジェクトが `ModelingExamples` 1件であることを確認し、必要なら右クリックで「スタートアッププロジェクトに設定」を選ぶ。
3. **Debug／Any CPU** を選び、**Ctrl+Shift+B** でビルドする。
4. **Ctrl+F5** でデバッグなし実行、**F5** でデバッグ実行する。最後に「すべての 153 チェックに成功しました。」と表示される。
5. **Release／Any CPU** に切り替えてビルド・実行する。内部の検証呼び出しはReleaseでも実行される。

F9／F11で生成過程を追う

1. Program.cs の `LocalPlacement` にある `V3 point = Place(frame, (2, -1))` に **F9** でブレークポイントを置く。**F5** で停止し、ローカル／ウォッチでC・N・B・Tを確認する。
2. **F11** で ModelingMath.cs の `Place` に入り、フレーム検証と `C+uN+vB` を追う。**Shift+F11** で戻り、pointが(10,22,29)になることを確認する。
3. `HomogeneousMatrix` の `Matrix4x4 row = NumericsRowMatrix(column)` にF9を置き、F11で列方式の16要素から転置する箇所へ進む。第4列のCが、行方式では第4行になることを確認する。
4. `ExtrusionVertices` の `CreateVertices` 呼び出しからF11で MeshBuilder.cs に入り、`vertices[i]` と `vertices[i + 4]` の代入を **F10** で進める。始端X=0と終端X=5、同じY・Zの8点ができる。

5. `FacesAndNormals` の `ExtrudeConvexQuad` 呼び出しへF9を移す。F11で `triangles.Add` の2行へ進み、i・jと、側面の順序(0,1,5)／(0,5,4)を追う。ループ後の4キャップも確認する。
6. `FaceGeometry geometry = AnalyzeTriangle(...)` からF11で入り、2辺の差→外積→2倍面積による退化検査→単位化の順を確認する。ゼロの外積を単位化しないことに注目する。
7. `ClosureAndVolume` の `HasPairedOppositeEdges` で辞書のCount・Balance、`SignedVolume` で基準点originと四面体の寄与を調べる。`StationFrames` では終点のNが－Xになること、角の左右で配置点が異なることを追う。

出力を読むにはCtrl+F5を使う。F5で終了前に止める場合はMainの成功メッセージにF9を置く。GUIでのF5／Ctrl+F5／F9／F11の操作はCIで未検証であり、CLIやMSBuildの成功とは区別する。

CLIでビルド・実行する

リポジトリのルートから実行する。

```
dotnet build sessions/04-modeling/examples/ModelingExamples.sln -c Debug
dotnet run --project sessions/04-modeling/examples/ModelingExamples.csproj -c Debug --no-build
dotnet build sessions/04-modeling/examples/ModelingExamples.sln -c Release
dotnet run --project sessions/04-modeling/examples/ModelingExamples.csproj -c Release --no-build
```

この `examples/` を作業ディレクトリにした場合は `dotnet run` でもよい。全件成功は終了コード0。不一致・非有限な比較値・想定外の例外・拒否されない無効入力は `FAIL` を表示して終了コード1になる。数値比較の失敗ではラベル・成分番号・実値・期待値・許容誤差を表示する。

ファイルと読む順序

ファイル	内容
ModelingExamples.sln	Visual Studioで開く入口。専用プロジェクト1件
ModelingExamples.csproj	net10.0のコンソール。追加パッケージなし
Program.cs	固定入力、独立な期待値、7題材、出力と実行時チェック
ModelingMath.cs	doubleの2D／3Dタプル、フレーム、成分式、列行列、行方式への変換、符号付き断面積
MeshBuilder.cs	頂点・三角形・面の計算結果の型、凸四角形の押し出し、辺の接続と符号付き体積

`Program.Main` → 固定入力 → 配置式 → 頂点配列 → 三角形配列 → 法線と面積 → 接続と体積の順に読む。メソッドにカーソルを合わせると、日本語XMLコメントで入力・結果・制約を確認できる。

題材・固定入力・期待結果

題材	解説／宿題との対応	主な入力・期待値	チェック数
1. LocalPlacement	②／基本1-1,2	A：C=(10,20,30)、N=+Y、B=+Z、T=+X、局所(2,－1) → (10,22,29)。直交単位基底、API照合	8
2. HomogeneousMatrix	②／基本1-3,4	N,B,T,Cの列方式。w=0・同次成分1 → (10,22,29,1)。行方式へ転置してVector3/4と一致	8
3. ExtrusionVertices	③／基本2-1,4	B：局所(－1,－0.5)→(1,－0.5)→(1,0.5)→(－1,0.5)。始端X=0、終端X=5の8点	9
4. FacesAndNormals	④／基本3-1,2	12三角形・36インデックス。全12枚の順序、3点、外積、単位法線、面積と外向き	62
5. ClosureAndVolume	⑤／基本2-2,3・基本3-3,4	断面積2×長さ5=10、辺18本の逆向き共有、独立な符号付き体積10。表面積34は別の値	7
6. StationFrames	⑥・⑦／発展	C：始点(0,1,0)、終点(4,4,0)、姿勢固定なら(5,5,0)、角の片側配置。D：回転点(0,2,1)	11
7. InvalidInputsAndPrecision	⑧／基本3・発展の注意	退化・逆順・非有限値・桁あふれ・開口を拒否。全反転の体積－10、floatで微小な差が消える	48

出力例（抜粋）：

```
  [OK] A: C+2N-B: 10, 22, 29
  [OK] A: 転置した行方式Vector4: 10, 22, 29, 1
  [OK] B: 頂点0: 0, -1, -0.5
  [OK] B: 頂点7: 5, -1, 0.5
  [OK] B: 三角形数: 12
  [OK] B: インデックス数: 36
  [OK] B: 三角形0の順序: 0, 1, 5
    3点: (0,-1,-0.5) / (0,1,-0.5) / (5,1,-0.5)
  [OK] B: 三角形0の外積: 0, 0, -10
  [OK] B: 三角形0の単位法線: 0, 0, -1
...
  [OK] B: 断面積×法線方向の長さ: 10
  [OK] B: 符号付き体積: 10
  [OK] C: 終点の局所(1,0): 4, 4, 0
...
すべての 153 チェックに成功しました。
```

チェックは未丸めの値を比較する。表示は通常G9、失敗時はG17とする。`-0` は浮動小数点の符号付きゼロで、期待する0と数値的に等しい。

座標・頂点・面の規約

- 右手系XYZ。V2は局所(u,v)、V3は世界(X,Y,Z)のdoubleタプル。点と方向を同じ型で表すため、文脈とコメントで区別する。
- N・Bは断面内の直交単位基底、T=N×B。輪郭は **+T側から断面の原点へ見る反時計回り** とする。u右・v上の局所座標に対応する。
- 始端i=0〜3、終端i+4=4〜7。`Vertices` は共有座標、`Triangles` は0始まりの順序付き3番号。三角形の各点は `Vertices[face.A]` などで取得する。
- 輪郭の次をj=(i+1) mod 4として、側面は(i,j,j+4)と(i,j+4,i+4)。内部を左に見る輪郭の辺eに対し、e×Tが右側の外向きになる。
- 始端キャップは(0,2,1)・(0,3,2)で−T、終端は(4,5,6)・(4,6,7)で+T。対応頂点の保存順は変えず、キャップの順序を変える。
- 外積 `(b-a)×(c-a)` の長さは三角形面積の2倍。退化検査を終えてから単位化する。底面の外積(0,0,−10)と単位法線(0,0,−1)を混同しない。

列方式のdouble配列は基底N,B,Tを列に、Cを第4列に置く。断面点の局所w=0と同次成分1は別物である。`System.Numerics` は行ベクトルvMの規約なので、`NumericsRowMatrix` は **Matrix4x4.Transposeで転置してから** `Vector3.Transform`／`Vector4.Transform` に渡す。公式の行方式の説明 と照合できる。

APIの入力制約と結果の意味

`CreateVertices`／`ExtrudeConvexQuad` は、**有限な4点、重複なし、反時計回りの厳密に凸な輪郭、右手系の直交単位基底、正の押し出し長**を要求する。非凸・自己交差・穴あり・一般多角形の三角形分割は扱わない。今回の8頂点・12三角形を任意のCAD形状へ一般化しない。

輪郭は先頭点を末尾に重複保存しない。配列だけでは最後の辺を閉じる意図が分からないため、`closed` を明示する。trueで末尾→先頭を接続し、falseは拒否する。始点を重複保存する形式の自動変換はしない。

入力制約違反、非有限な中間値、下限以下の形状には `ArgumentException` を投げ、偽の原点・法線を返さない。`CreateVertices` は配置段階、`ExtrudeConvexQuad` はさらに生成した全三角形を検査する段階である。大きな世界座標で異なる局所点が同じ座標へ丸められる場合も、後者の退化検査で拒否する。一般の丸め誤差をすべて検出する保証はない。

`HasPairedOppositeEdges` は各無向辺のCount=2と向きのBalance=0を検査する。片方のキャップが欠けた場合や1枚だけの反転は検出するが、全反転は通る。**辺の対だけで自己交差・頂点周りの非多様体接続・外向き・有効ソリッドを保証しない。** 三角形の幾何も別に検査する。

`SignedVolume` は辺の対と全三角形の非退化を検査してから、基準点oに対する符号付き四面体 `(a-o)・((b-o)×(c-o))/6` を足す。oは最初の頂点とし、大きな共通平行移動による相殺を減らす。本文の世界原点の式と、閉じた境界では同じ体積になる。教材では単純な凸直柱の検算に限り、まず断面積×法線方向の長さ

で10と確認する。各寄与の絶対値を足さず、全体を反転したら−10になることも確認する。

メッシュは有限個の面による境界表現で、この直柱の平面は正確に表せるが、曲面は通常近似する。CADのB-repソリッドを生成・検証するものではない。経路の2フレームの配置と回転点は点の検算に限り、コーナー接続・輸送フレーム・汎用スイープ／回転体メッシュの生成は含めない。

数値精度としきい値

条件	教材の設定・尺度
数値の期待値比較	doubleは絶対誤差1e-12、floatのAPIは1e-5。実値・期待値の両方の有限性も検査
押し出し長・局所点間距離	MinimumLength=1e-12以下を拒否。長さ単位
角の2D外積・三角形の2倍面積	MinimumArea2=1e-12以下を拒否。長さの2乗単位
フレーム	内積・長さの2乗と1の差・N×B−Tの長さにFrameTolerance=1e-12。無次元

これらは固定単位・小さな入力向けの設定で、厳密な幾何判定ではない。長さ、面積、無次元の角度条件へ同じしきい値を無条件に流用しない。単位や尺度を変える際は設定も見直す。桁あふれ・アンダーフロー、座標差の桁落ち、境界付近の誤差への汎用的な対策は範囲外である。

`System.Numerics` の成分は Single（float） で、doubleから渡すと丸められる。題材7の `1+1e-10` の差はdoubleに残るが、floatでは消える。doubleに戻しても復元できない。許容誤差内でも元の情報が完全に保存されたとは限らない。

自動チェックとCI

実際に成功した `Check`／`Assert`／`Reject` の呼び出しを各1件として数える。複数成分の比較は1件、入力反復の各呼び出しは別件とする。7題材の件数8／8／9／62／7／11／48の合計153、実際の `[OK]` 153行、成功要約、終了コード0が揃うことを確認する。

Check Modeling Examples は、第4回のC#コード（`.cs`）・プロジェクト（`.csproj`）・ソリューション（`.sln`）・ビルド設定（`.props`／`.targets`）、ルートの `global.json`／`Directory.Build.props`／`Directory.Build.targets`、ワークフロー自身の変更を、PR時とmainへのpush時に検証する。Markdownだけの変更では起動しない。

Ubuntu 24.04・.NET SDK 10.0.401でソリューションをrestoreし、Releaseビルドして実行する。終了コード0、実際の `[OK]` 153行、期待する成功要約の3点を確認する。`set -euo pipefail` により、ログを `tee` で表示・保存している途中の実行失敗も伝播する。チェック内容を変更する場合は、題材をレビューし、このREADMEの件数・出力例とワークフローの `EXPECTED_CHECKS` を同じPRで更新する。

結果は Actions → Check Modeling Examples、またはPRのChecksから開く。**対象の実行 → ModelingExamples (Release)** の `Restore solution`・`Build solution (Release)`・`Run ModelingExamples and verify all 153 checks` のログを確認する。

失敗時は `FAIL` の検証ラベルから、押し出しの頂点・三角形の順序・外積・体積などの実値と期待値を追う。数値比較では成分番号と許容誤差も表示する。件数や成功要約が一致しない場合は `::error::` が表示される。正常時には全12三角形の法線・面積、辺の共有、体積10のチェックと、最後の `Verified all 153 checks.` を確認できる。

権限は `contents: read` のみで、チェックアウトした認証情報を保持せず、Secret・PAT・書き込み権限は不要。ソース・OBJ・PDFの生成コミットは行わない。第1回43件・第2回73件・第3回155件の専用CI、PDF自動生成から独立したワークフローである。

資料の元データ（GitHubリポジトリ）： https://github.com/mkunori/geometry-study-group